techcraft1999@gmail.com

TECH CRAFT

AI AGENTS

Comprehensive Study Notes & Interview Preparation Guide

Day 6 — Complete Training Material
Beginner to Advanced | Industry-Oriented | Interview-Ready

Covers: Definitions | Architecture | Tool Calling | LangChain | Code | Interview Q&A

1. What Is an AI Agent?

1.1 Definition

An AI Agent is an intelligent, autonomous software system designed to perceive its environment, reason about what it observes, make decisions based on its goals, and take actions — all with minimal or no human intervention. Unlike traditional software that executes a fixed sequence of instructions, an AI Agent is dynamic. It can adapt its behavior based on new information and use external tools to accomplish tasks in the real world.

> **CORE DEFINITION**
> AI Agents are autonomous systems that think, plan, and act to complete tasks. They combine the reasoning ability of Large Language Models (LLMs) with external tools and APIs to solve complex, multi-step problems automatically.

1.2 The Four Pillars of an AI Agent

Every AI Agent is built around four core behaviors:

- OBSERVE: The agent takes in inputs from the environment. This can be a text message, an API response, a document, sensor data, or any structured/unstructured information.
- REASON/DECIDE: The agent uses a Large Language Model (LLM) to understand the input, identify the user's intent, break down the problem into steps, and decide what action to take next.
- ACT: The agent executes actions — this can include calling APIs, running code, fetching data, sending emails, filling forms, booking appointments, and much more.
- ACHIEVE GOALS: The agent repeats the above cycle until the desired goal or task is complete, adjusting its behavior if needed.

1.3 AI Agent vs Traditional Automation vs Simple Chatbot

Feature	Traditional Software	AI Agent
Flexibility	Fixed logic / rules	Adapts dynamically based on context
Decision Making	Pre-programmed paths	LLM-powered reasoning
Tool Use	Hardcoded integrations	Can discover and call tools autonomously
Multi-step Tasks	Requires manual orchestration	Handles complex workflows automatically
Goal Orientation	Executes commands	Works toward achieving end goals
Error Handling	Fails or returns error	Can retry, adapt, or ask for clarification

1.4 Why AI Agents Matter — Industry Perspective

AI Agents represent a fundamental shift in how software systems are built. Traditionally, if you wanted to automate a multi-step process (like booking a flight), you needed to write custom code for every possible scenario. With AI Agents, you describe the goal to an LLM, give it the right tools, and it figures out the steps on its own.

This dramatically reduces development time and enables automation of tasks that previously required human judgment. According to industry analysts, agentic AI is one of the fastest-growing areas in enterprise software, with companies investing heavily in agent frameworks, toolchains, and infrastructure.

1.5 Real-World Analogy

ANALOGY — The Smart PA
Think of an AI Agent as a highly intelligent personal assistant. You tell them: 'Book me a flight to Delhi next Friday and email me the details.' They understand your request, check your calendar, search for available flights, compare prices, book the best option, and email you the confirmation — all without you giving step-by-step instructions. That's exactly what an AI Agent does.

2. Applications of AI Agents

AI Agents are being deployed across virtually every industry. Below are the major application domains with detailed real-world use cases.

2.1 Customer Support

Customer support is one of the earliest and most impactful domains for AI Agents. Traditional chatbots were limited to pre-scripted answers. Modern AI Agents can hold context-aware, multi-turn conversations and actually resolve issues end-to-end.

- AI-powered chatbots that understand natural language and handle complex queries
- Automated ticket triage and routing to the right department
- Real-time order tracking, refund processing, and account management
- Escalation handling — the agent knows when to involve a human agent
- Multi-language support through integrated translation tools

2.2 Healthcare

Healthcare agents assist both patients and medical professionals by automating administrative tasks and providing intelligent decision support.

- Patient onboarding, symptom collection, and preliminary assessment
- Appointment scheduling and reminder management via calendar APIs
- Automated medical report analysis (summarizing diagnostic reports)
- Drug interaction checks and prescription validation
- Insurance claim pre-authorization and document verification

2.3 Software Development

AI Agents are revolutionizing how developers write, test, and debug code. Tools like GitHub Copilot, Cursor, and Claude Code are examples of agentic systems in software development.

- Code generation from natural language requirements
- Automated code review — checking for bugs, security issues, style violations
- Debugging assistants that identify root causes and suggest fixes
- Test case generation and automated testing pipelines
- Documentation generation from source code
- Dependency management and upgrade recommendations

2.4 Business Automation

Enterprises are using AI Agents to automate complex business processes that previously required significant human effort.

- Intelligent scheduling — coordinating meetings across time zones
- Automated data analysis — generating insights from raw datasets
- Report generation — pulling data from multiple sources and creating formatted reports
- Workflow orchestration — managing multi-step approval processes
- Email management — sorting, drafting replies, and flagging urgent messages
- Financial reconciliation — matching transactions and flagging discrepancies

Industry	Example AI Agent Use Case
E-commerce	Product recommendation agent, return processing agent
Banking	Fraud detection agent, loan eligibility assessment agent
Legal	Contract review agent, legal research agent
HR	Resume screening agent, onboarding assistant agent
Education	Personalized tutoring agent, exam preparation agent
Real Estate	Property search agent, market analysis agent

3. Basic Architecture of an AI Agent

3.1 The Core Pipeline

Every AI Agent, regardless of complexity, follows a fundamental pipeline. Understanding this pipeline is essential for building and debugging agents.

THE AGENT PIPELINE
User Input → LLM (Brain) → Tool Selection → Tool Execution (APIs/Functions) → Result Processing → Final Output to User

3.2 Component Deep Dive

Component	Role & Responsibility
User Input	The starting point. Can be a text message, voice command, uploaded file, or event trigger. The input defines the task the agent must complete.
LLM (Brain)	The Large Language Model (GPT-4, Claude, Gemini, etc.) is the reasoning engine. It understands the input, breaks it into sub-tasks, decides which tools to use, and generates the final response.
Tools / APIs	External functions the LLM can call to interact with the real world. Examples: weather APIs, database queries, email senders, web browsers, file systems, calculators.
Decision Layer	Logic that determines the agent's next action based on tool results. In multi-step tasks, the agent may call multiple tools in sequence before reaching the final answer.
Memory (Optional)	Short-term memory (conversation history) keeps context within a session. Long-term memory (vector databases) allows the agent to recall past interactions.
Output	The final response delivered to the user, which may include text, structured data, emails sent, files created, bookings confirmed, etc.

3.3 The ReAct Pattern (Reason + Act)

Most modern AI Agents use a pattern called ReAct — short for Reason + Act. This means the LLM alternates between reasoning (thinking out loud about what to do next) and acting (calling a tool and observing the result). This cycle continues until the task is complete.

```
ReAct Loop:

1. Thought:  'I need to find the weather in Ahmedabad.'
2. Action:   Call get_weather(city='Ahmedabad')
3. Observe:  Tool returns: 'Temperature is 38°C with Clear sky'
```

4. Thought: 'I now have the weather. I can respond.'
5. Final: 'The current weather in Ahmedabad is 38°C with clear skies.'

3.4 Types of AI Agents by Complexity

Agent Type	Description & Example
Simple Reflex Agent	Acts based only on current input. No memory. Example: A thermostat that turns heating on/off based on temperature sensor.
Model-Based Reflex Agent	Maintains internal state (memory of past events). Example: A navigation agent that remembers which roads were already explored.
Goal-Based Agent	Works toward explicit goals, planning multiple steps ahead. Example: A travel booking agent that plans the cheapest route.
Utility-Based Agent	Maximizes a utility function (score of how 'good' a state is). Example: A trading agent that maximizes portfolio returns.
Learning Agent	Improves performance over time through feedback. Example: A recommendation agent that learns from user preferences.
Multi-Agent System	Multiple agents working together. Example: A customer support system with specialized agents for billing, technical issues, and escalations.

4. Tool Calling in AI Agents

4.1 What Is Tool Calling?

Tool calling (also called function calling) is the mechanism by which an LLM can invoke external functions, APIs, or services during its reasoning process. Instead of generating a final text answer, the LLM outputs a structured request to call a specific function with specific arguments. The calling system then executes that function and feeds the result back to the LLM.

WHY TOOL CALLING IS REVOLUTIONARY
Without tool calling, LLMs are limited to information from their training data. Tool calling gives agents access to real-time data (weather, stock prices), external services (email, databases), and computational power (running code, math operations). This transforms a text predictor into a capable autonomous agent.

4.2 How Tool Calling Works — Step by Step

1. You define one or more functions (tools) with clear descriptions and parameter types.
2. You bind these tools to an LLM instance, so the model knows they are available.
3. When the user sends a query, the LLM decides whether a tool is needed.
4. If yes, the LLM outputs a tool_call object with the function name and arguments — it does NOT make the call itself.
5. Your code detects this tool_call, executes the function, and sends the result back to the LLM.
6. The LLM uses the tool result to formulate the final response.

4.3 Building a Tool — The Weather Function Example

Let's build a complete, beginner-friendly weather tool from scratch. We will use the free wttr.in API.

Step 1: Import Required Libraries

```
import requests   # Used to make HTTP requests to external APIs
                  # Install: pip install requests
```

Step 2: Define the Tool Function

```
def get_weather(city: str) -> str:
    """
    Returns current weather of a city.
    This docstring is important — the LLM reads it to understand what the tool does.

    Args:
        city (str): The name of the city to get weather for.
                    Example: 'Ahmedabad', 'Mumbai', 'Delhi'

    Returns:
```

```
    str: A human-readable weather summary.
  """
  # Build the API URL using f-string formatting
  # ?format=j1 requests JSON response from wttr.in
  url = f'https://wttr.in/{city}?format=j1'

  # Make the GET request to the weather API
  response = requests.get(url)

  # Parse the JSON response into a Python dictionary
  data = response.json()

  # Navigate the nested JSON to extract temperature in Celsius
  # data['current_condition'] is a list; [0] gets the first (current) entry
  temp = data['current_condition'][0]['temp_C']

  # Extract the weather description (e.g., 'Sunny', 'Partly Cloudy')
  condition = data['current_condition'][0]['weatherDesc'][0]['value']

  # Return a formatted string with the result
  return f'Temperature in {city} is {temp}°C with {condition}'
```

Step 3: Test the Function Directly

```
# Direct call to test the function works before connecting to LLM
result = get_weather('Ahmedabad')
print(result)

# Expected Output:
# Temperature in Ahmedabad is 38°C with Sunny
```

4.4 JSON Response Structure of wttr.in

Understanding the API response helps you navigate and extract data correctly:

```
{
 "current_condition": [
   {
    "temp_C": "38",                    // Temperature in Celsius (string)
    "temp_F": "100",                   // Temperature in Fahrenheit
    "humidity": "15",                  // Humidity percentage
    "weatherDesc": [{"value": "Sunny"}],    // Weather description
    "windspeedKmph": "25",             // Wind speed in km/h
    "uvIndex": "8"                     // UV index
   }
 ]
}
```

4.5 Enhanced Weather Tool — Adding Error Handling

```
def get_weather(city: str) -> str:
   """Returns current weather of a city with error handling."""
   try:
```

```
    url = f'https://wttr.in/{city}?format=j1'
    response = requests.get(url, timeout=10)   # timeout prevents hanging forever

    # Check if the request was successful (status code 200)
    response.raise_for_status()

    data = response.json()
    temp = data['current_condition'][0]['temp_C']
    feels_like = data['current_condition'][0]['FeelsLikeC']
    humidity = data['current_condition'][0]['humidity']
    condition = data['current_condition'][0]['weatherDesc'][0]['value']

    return (
        f'Weather in {city}:\n'
        f'  Temperature: {temp}°C (feels like {feels_like}°C)\n'
        f'  Condition: {condition}\n'
        f'  Humidity: {humidity}%'
    )

except requests.exceptions.Timeout:
    return f'Error: Request timed out for city {city}'
except requests.exceptions.HTTPError as e:
    return f'Error: City not found or API error — {e}'
except KeyError:
    return f'Error: Unexpected response format from weather API'
```

5. Binding Tools with LangChain

5.1 What Is LangChain?

LangChain is an open-source Python framework designed to simplify the development of applications powered by Large Language Models. It provides abstractions for working with LLMs, chaining operations together, managing memory, and connecting to external tools and data sources. LangChain is the most widely used framework for building AI Agents.

LangChain Feature	What It Does
LLM Wrappers	Unified interface to OpenAI, Anthropic, Cohere, Hugging Face models
@tool decorator	Converts any Python function into a LangChain-compatible tool
bind_tools()	Connects a list of tools to an LLM instance
Chains	Sequence multiple LLM calls and operations together
Memory	Manage conversation history and context
Agents	Create fully autonomous agents with tool use and planning
LCEL	LangChain Expression Language — declarative chain syntax

5.2 Installation

```
# Install all required packages
pip install requests langchain langchain-openai python-dotenv

# Package purposes:
# requests       — HTTP requests for APIs
# langchain      — Core LangChain framework
# langchain-openai — OpenAI integration for LangChain
# python-dotenv  — Load environment variables from .env file
```

5.3 Environment Setup — API Key Management

Never hardcode API keys in your code. Use environment variables and the python-dotenv library.

```
# Step 1: Create a .env file in your project root
# .env file contents:
OPENAI_API_KEY=sk-your-api-key-goes-here

# Step 2: Add .env to your .gitignore
echo '.env' >> .gitignore

# Step 3: Load variables in Python
from dotenv import load_dotenv
import os
```

```
load_dotenv()   # Reads the .env file and loads variables into environment

# Now access the API key
api_key = os.getenv('OPENAI_API_KEY')   # Returns None if not set
```

5.4 The @tool Decorator — Converting Functions to Tools

The @tool decorator is LangChain's way of converting a regular Python function into a tool that an LLM can recognize and call. When you apply @tool, LangChain wraps the function and extracts its name, docstring, and parameter types to create a ToolSchema — a structured description the LLM uses to decide when and how to call the tool.

```
import requests
from langchain.tools import tool
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

# Load API keys from .env file
load_dotenv()

# The @tool decorator registers this function as a LangChain tool
# The LLM reads the function name and docstring to understand what it does
@tool
def get_weather(city: str) -> str:
    """
    Returns the current weather for a given city.
    Use this tool when the user asks about weather in any location.
    Input: city name as a string (e.g., 'Ahmedabad', 'Delhi')
    Output: Weather description including temperature and conditions.
    """
    url = f'https://wttr.in/{city}?format=j1'
    response = requests.get(url)
    data = response.json()
    temp = data['current_condition'][0]['temp_C']
    condition = data['current_condition'][0]['weatherDesc'][0]['value']
    return f'Temperature in {city} is {temp}°C with {condition}'

# Print tool metadata to see what the LLM receives
print(get_weather.name)         # Output: get_weather
print(get_weather.description)   # Output: Returns the current weather...
print(get_weather.args)         # Output: {'city': {'title': 'City', 'type': 'string'}}
```

5.5 Binding Tools to an LLM

bind_tools() is a method that connects your list of tools to a specific LLM instance. Once tools are bound, the LLM is aware of these tools and can decide to call them when processing a user query.

```
# Create an LLM instance
# model: specifies which OpenAI model to use
# temperature: controls randomness (0 = deterministic, 1 = creative)
llm = ChatOpenAI(
    model='gpt-4o-mini',   # Lightweight and cost-effective model
    temperature=0          # Use 0 for reliable tool calling behavior
```

```
)

# Bind the weather tool to the LLM
# You can pass multiple tools in the list: [tool1, tool2, tool3]
llm_with_tools = llm.bind_tools([get_weather])

# llm_with_tools now behaves like the LLM but with tool awareness
# When it encounters a weather-related query, it can invoke get_weather
```

5.6 Executing Queries and Handling Tool Calls

After binding tools, you invoke the LLM with a query. The response may either be a direct text answer or a tool_call request. Your code must handle both cases.

```
# Define the user query
query = 'What is the weather in Ahmedabad?'

# Invoke the LLM with the query
# The LLM processes the query and decides: text response OR tool call
response = llm_with_tools.invoke(query)

# Check if the LLM wants to call a tool
# response.tool_calls is a list of tool call requests
# If empty [] or None — the LLM responded with text directly
if response.tool_calls:

    # Get the first tool call (there could be multiple)
    tool_call = response.tool_calls[0]

    # Extract the name of the tool the LLM wants to call
    tool_name = tool_call['name']
    print(f'LLM wants to call: {tool_name}')   # Output: get_weather

    # Extract the arguments the LLM wants to pass to the tool
    tool_args = tool_call['args']
    print(f'With arguments: {tool_args}')       # Output: {'city': 'Ahmedabad'}

    # Actually execute the tool with the provided arguments
    # .invoke() runs the function and returns its result
    result = get_weather.invoke(tool_args)
    print(result)    # Output: Temperature in Ahmedabad is 38°C with Sunny

else:
    # LLM answered directly without needing a tool
    print(response.content)
```

5.7 Complete Working Example — Multi-Tool Agent

Here is a more complete example with multiple tools and proper error handling:

```
import requests
from langchain.tools import tool
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
```

```
load_dotenv()

@tool
def get_weather(city: str) -> str:
    """Returns current temperature and weather condition for a city."""
    url = f'https://wttr.in/{city}?format=j1'
    try:
        data = requests.get(url, timeout=10).json()
        temp = data['current_condition'][0]['temp_C']
        cond = data['current_condition'][0]['weatherDesc'][0]['value']
        return f'{city}: {temp}°C, {cond}'
    except Exception as e:
        return f'Could not fetch weather: {e}'

@tool
def calculate(expression: str) -> str:
    """Evaluates a basic mathematical expression. Example: "2 + 2", "100 / 4"."""
    try:
        result = eval(expression, {'__builtins__': {}})  # Safe eval
        return f'Result: {result}'
    except Exception as e:
        return f'Calculation error: {e}'

# Register both tools
tools = [get_weather, calculate]
llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
llm_with_tools = llm.bind_tools(tools)

# Tool registry for dynamic dispatch
tool_registry = {t.name: t for t in tools}

def run_agent(user_query: str):
    print(f'User: {user_query}')
    response = llm_with_tools.invoke(user_query)

    if response.tool_calls:
        for tool_call in response.tool_calls:
            name = tool_call['name']
            args = tool_call['args']
            if name in tool_registry:
                result = tool_registry[name].invoke(args)
                print(f'Tool [{name}]: {result}')
    else:
        print(f'Agent: {response.content}')

# Test the agent
run_agent('What is the weather in Mumbai?')
run_agent('What is 15% of 240?')
run_agent('Tell me a fun fact about Python.')
```

6. Common AI Agent Workflow

6.1 The Standard Agent Loop

The following workflow describes how a production-grade AI Agent processes a request from start to finish. Each step has important implementation considerations.

AGENT WORKFLOW OVERVIEW
Step 1: User sends a query Step 2: LLM understands intent and plans actions Step 3: Agent selects the appropriate tool Step 4: Tool / API executes the real-world task Step 5: Result is returned to the LLM for processing Step 6: LLM generates the final response for the user

6.2 Travel Booking Agent — Full Workflow Example

Let's trace through a complete example to understand how each component works:

User Request: 'Book me a flight to Delhi next Friday'

Step	What Happens Inside the Agent
1. Input Received	The agent receives the text 'Book me a flight to Delhi next Friday'. This is passed to the LLM as the user message.
2. Intent Analysis	The LLM identifies key entities: Destination = Delhi, Date = next Friday. It determines the task requires multiple tool calls: date resolver, flight search, booking confirmation, email sender.
3. Date Resolution	Agent calls get_current_date() tool to determine today's date. Calculates 'next Friday' relative to today's date.
4. Flight Search	Agent calls search_flights(origin='Ahmedabad', destination='Delhi', date='2026-06-05') and receives a list of available flights.
5. Option Selection	LLM evaluates the returned flights based on time and price. Selects the best option based on user preferences (or asks user to choose).
6. Booking	Agent calls book_flight(flight_id='AI302', passenger_name='User') and receives a booking confirmation code.
7. Email Confirmation	Agent calls send_email(to='user@example.com', subject='Flight Booking', body='Your booking is confirmed...').
8. Final Response	LLM generates a natural language summary: 'Done! I have booked Air India flight AI302 to Delhi on June 5th. Your confirmation code is XYZ123. A confirmation has been sent to your email.'

6.3 Implementing a Simple Workflow Agent

```
from datetime import datetime, timedelta

# Simulated tools for demonstration
@tool
def get_current_date() -> str:
    """Returns today's date in YYYY-MM-DD format."""
    return datetime.now().strftime('%Y-%m-%d')

@tool
def search_flights(origin: str, destination: str, date: str) -> str:
    """
    Searches for available flights.
    Args:
        origin: Departure city
        destination: Arrival city
        date: Travel date in YYYY-MM-DD format
    """
    # In production, this would call a real flight API
    return (
        f'Found 3 flights from {origin} to {destination} on {date}:\n'
        '1. AI302 - 06:00 AM - Rs. 4,500\n'
        '2. 6E205 - 10:30 AM - Rs. 3,800\n'
        '3. SG110 - 03:00 PM - Rs. 3,200'
    )

@tool
def book_flight(flight_id: str, passenger_name: str) -> str:
    """Books a specific flight for the passenger."""
    import random
    confirmation = f'BOOK{random.randint(10000, 99999)}'
    return f'Flight {flight_id} booked for {passenger_name}. Confirmation: {confirmation}'
```

7. Key Concepts & Important Terms

7.1 Comprehensive Glossary

Term	Explanation
LLM (Large Language Model)	A deep learning model trained on massive text datasets. Capable of understanding and generating human-like text. Examples: GPT-4, Claude 3, Gemini 1.5, Llama 3.
Tool Calling / Function Calling	A mechanism that allows an LLM to request execution of external functions. The LLM outputs a structured JSON object specifying the function name and arguments.
Agent	An autonomous AI system that uses an LLM for reasoning and tools for acting. Can complete multi-step tasks without step-by-step human guidance.
API (Application Programming Interface)	A set of defined rules that enables different software applications to communicate. Agents use APIs to access external data and services (weather, databases, email, etc.).
Prompt Engineering	The practice of crafting clear, structured inputs to guide LLM behavior. Good prompts produce reliable, predictable agent behavior.
Context Window	The maximum amount of text an LLM can process in one request. GPT-4 has 128K tokens; Claude has up to 200K tokens.
Temperature	A parameter that controls LLM randomness. 0 = always picks the most likely token (deterministic); 1 = more creative and varied outputs. Use 0 for tool calling.
Token	The basic unit of text for LLMs. A token is roughly 4 characters or 0.75 words. API pricing is typically per token.
Chain	A sequence of LLM calls or operations connected together. LangChain's core abstraction for building multi-step pipelines.
Memory	The ability of an agent to retain information from past interactions. Short-term = conversation history; Long-term = vector databases.
Vector Database	A database that stores data as numerical vectors for semantic search. Used for agent long-term memory and RAG (Retrieval-Augmented Generation).
RAG	Retrieval-Augmented Generation. An architecture where the agent retrieves relevant documents from a knowledge base before answering questions.
Orchestration	The process of coordinating multiple agents, tools, and workflows to complete complex tasks. Frameworks like LangGraph and CrewAI handle orchestration.
Hallucination	When an LLM generates confident but factually incorrect information. Tool calling helps ground agents in real-world data to reduce hallucinations.

7.2 Libraries Reference

Library	Purpose & Key Features
requests	Python HTTP library for making API calls. Methods: get(), post(), put(), delete(). Returns Response objects with .json(), .text, .status_code.
langchain	Core LangChain framework. Provides @tool decorator, agent classes, chains, memory, and prompt templates.
langchain-openai	LangChain integration for OpenAI models. Provides ChatOpenAI class for using GPT models with LangChain.
python-dotenv	Loads environment variables from a .env file. Critical for secure API key management.
langchain-anthropic	LangChain integration for Claude (Anthropic) models.
langchain-community	Community-contributed tools, loaders, and integrations.
langgraph	Graph-based agent orchestration framework. Enables complex multi-step agent workflows.

8. Best Practices, Tips & Common Mistakes

8.1 Best Practices for Building AI Agents

Tool Design Best Practices

- Write clear, specific docstrings — the LLM uses your docstring to decide when to call the tool. Vague descriptions lead to wrong tool selection.
- Keep tools focused — each tool should do one thing well. Avoid creating overly complex, multi-purpose tools.
- Use strong typing — always annotate function parameters and return types. This helps LangChain generate accurate tool schemas.
- Handle all exceptions inside the tool — never let a tool raise an unhandled exception. Always return error messages as strings.
- Validate inputs before processing — check that arguments are in the expected format before making external API calls.

LLM Configuration Best Practices

- Use temperature=0 for tool calling — deterministic behavior is critical for reliable agent performance.
- Choose the right model — GPT-4o for complex reasoning, GPT-4o-mini for speed and cost efficiency.
- Manage token limits — long conversations accumulate tokens. Implement conversation summarization for long sessions.
- Test with edge cases — try unusual inputs, invalid cities, empty queries. Agents must handle gracefully.

Security Best Practices

- Never hardcode API keys — always use environment variables and .env files.
- Add .env to .gitignore — prevent accidental commits of credentials.
- Validate and sanitize tool inputs — never pass raw user input to functions like eval() without restrictions.
- Implement rate limiting — avoid hammering external APIs and incurring costs or bans.
- Log all agent actions — maintain audit trails for debugging and compliance.

8.2 Common Beginner Mistakes

Mistake	Why It Happens & How to Fix It
Tool not being called	Docstring is too vague. Fix: Write a clear description that explicitly says when to use this tool.

Mistake	Why It Happens & How to Fix It
Wrong tool arguments	Parameter names are unclear. Fix: Use descriptive parameter names and document expected values.
API key errors	Key not loaded. Fix: Call load_dotenv() before creating LLM instance. Check .env file path.
KeyError on JSON response	Assuming API response structure. Fix: Print the raw response and navigate the actual structure.
Infinite loops	Agent keeps calling tools without converging. Fix: Set max_iterations limit on your agent.
High API costs	Calling GPT-4 for simple tasks. Fix: Use GPT-4o-mini or gpt-3.5-turbo for testing.
Ignoring tool_calls check	Processing response.content when tool was called. Fix: Always check if response.tool_calls before accessing content.
Not handling tool errors	Tool fails and crashes the agent. Fix: Wrap all tool code in try/except and return error strings.

8.3 Debugging AI Agents

Step-by-Step Debugging Approach

7. Test tools independently — call each tool function directly and verify it returns the expected output.
8. Print the raw LLM response — print(response) before checking tool_calls to see the full response object.
9. Check tool_calls content — print(response.tool_calls) to see exactly what function and arguments the LLM requested.
10. Verify API connectivity — test your external API calls separately with requests to rule out network issues.
11. Check environment variables — print(os.getenv('OPENAI_API_KEY')) to verify the key is loaded.
12. Add verbose logging — use LangChain's verbose=True flag to see the full chain of thoughts.

```
# Enable verbose mode for debugging
llm = ChatOpenAI(model='gpt-4o-mini', verbose=True)

# Print full response for debugging
response = llm_with_tools.invoke(query)
print('Full response:', response)        # Show everything
print('Tool calls:', response.tool_calls) # Show tool call details
print('Content:', response.content)      # Show text response
```

9. Practice Tasks & Coding Exercises

9.1 Beginner Level Tasks

Task 1 — Basic Tool Creation

Create a Python function called get_joke() that fetches a random programming joke from the Official Joke API (https://official-joke-api.appspot.com/jokes/programming/random) and returns the setup and punchline as a formatted string.

```
# Expected Output:
# Q: Why do Java developers wear glasses?
# A: Because they don't C#!


# Hints:
# 1. Use requests.get() to fetch from the API
# 2. The response is a list: data[0]['setup'] and data[0]['punchline']
```

Task 2 — Tool with Parameters

Create a currency_converter(amount: float, from_currency: str, to_currency: str) -> str tool that converts between currencies. Use a mock exchange rate dictionary for now.

```
# Mock exchange rates (relative to USD)
RATES = {
   'USD': 1.0,
   'INR': 83.5,
   'EUR': 0.92,
   'GBP': 0.79,
}
# Expected Output: '100 USD = 8350.0 INR'
```

Task 3 — Error Handling

Modify your get_weather() function to gracefully handle the following error cases and return user-friendly messages: (a) City name is empty string, (b) API request times out, (c) City is not found (API returns an error), (d) Network is unavailable.

9.2 Intermediate Level Tasks

Task 4 — Multi-Tool Agent

Build an agent with three tools: get_weather(), calculate(), and get_joke(). Create a run_agent() function that routes queries to the correct tool. Test with at least 5 different queries covering all three tools.

Task 5 — Tool with Multiple Return Values

Enhance the weather tool to return: temperature, humidity, wind speed, UV index, and a 3-day forecast. Structure the output clearly for both human reading and LLM processing.

Task 6 — Simulated Database Tool

Create a student_lookup(student_id: str) -> str tool that returns student information (name, grade, attendance) from a Python dictionary. Connect it to an LLM and test queries like 'What is the grade of student S001?'.

```
STUDENT_DB = {
    'S001': {'name': 'Raj Patel', 'grade': 'A', 'attendance': '92%'},
    'S002': {'name': 'Priya Shah', 'grade': 'B+', 'attendance': '88%'},
    'S003': {'name': 'Arjun Mehta', 'grade': 'A+', 'attendance': '97%'},
}
```

9.3 Advanced Level Tasks

Task 7 — Sequential Tool Calling

Build a trip planner agent that: (1) Gets weather for the destination city, (2) Calculates the trip budget based on given daily rate and number of days, (3) Returns a formatted trip summary combining both results.

Task 8 — Mini Project: Personal Assistant Agent

Build a Personal Assistant Agent with the following tools: get_weather(), calculate(), get_current_date(), set_reminder(task: str, time: str), and get_news_summary(topic: str). The agent should handle multi-intent queries like 'What is the weather today and remind me to take medicine at 6 PM?'.

Task 9 — Scenario-Based Problem

Your company receives 500 customer support emails per day. Design an AI Agent system that: reads each email, classifies it (billing, technical, general), extracts key information (customer ID, issue), routes it to the right department, and sends an automated acknowledgement. Define the tools, the workflow, and explain how the agent would handle edge cases.

10. Interview Preparation — Q&A Guide

10.1 Beginner Level Interview Questions

Q1: What is an AI Agent? How is it different from a regular chatbot?

ANSWER
An AI Agent is an autonomous system that can observe its environment, reason using an LLM, and take actions using external tools to accomplish goals. A regular chatbot is reactive — it responds to inputs with pre-defined or LLM-generated text but cannot take actions in the real world. An AI Agent can browse the web, call APIs, book appointments, send emails, and complete multi-step tasks autonomously. The key differentiator is tool use and autonomous action.

Q2: What does the @tool decorator do in LangChain?

ANSWER
The @tool decorator converts a regular Python function into a LangChain-compatible Tool object. It extracts the function name, docstring, and parameter type annotations to create a ToolSchema — a structured description that the LLM uses to understand when and how to call the tool. Without this decorator, the LLM has no knowledge of the function's existence or purpose.

Q3: What is bind_tools() and why is it important?

ANSWER
bind_tools() is a LangChain method that registers a list of tools with an LLM instance. When you call llm.bind_tools([tool1, tool2]), the tool schemas are sent to the LLM in every request, making the LLM aware of what tools are available. Without bind_tools(), the LLM would not know any tools exist and would only generate text responses.

Q4: What is response.tool_calls and when does it have a value?

ANSWER
response.tool_calls is a list of tool call requests returned by the LLM when it determines a tool needs to be used to answer the user's query. It contains the function name and the arguments to pass to it. It is non-empty only when the LLM decides a tool call is needed. If the LLM can answer from its knowledge, response.tool_calls will be empty and response.content will contain the text answer.

Q5: What Python libraries are needed to build a basic AI Agent with LangChain?

ANSWER
requests (for making HTTP API calls), langchain (core framework with @tool decorator and agent classes), langchain-openai (for using OpenAI GPT models with LangChain), and python-dotenv (for secure API key management using .env files). Install with: pip install requests langchain langchain-openai python-dotenv

10.2 Intermediate Level Interview Questions

Q6: Explain the ReAct pattern in AI Agents.

ANSWER
ReAct (Reason + Act) is a prompting/architecture pattern where the agent alternates between Thought (reasoning about what to do next) and Action (calling a tool and observing the result). This cycle repeats until the task is complete. The key advantage is that the agent's reasoning is explicit and observable, making it easier to debug. LangChain's AgentExecutor uses this pattern internally.

Q7: How do you handle errors in AI Agent tools?

ANSWER
All tool functions should be wrapped in try/except blocks. Instead of raising exceptions, tools should return error messages as strings. This way, the LLM receives the error message as a tool result and can decide how to handle it — it might retry with different parameters, inform the user, or try an alternative tool. Example: except Exception as e: return f'Error: {e}'. This prevents the entire agent from crashing due to a single tool failure.

Q8: What is the difference between short-term and long-term memory in AI Agents?

ANSWER
Short-term memory is the conversation history maintained within a single session. It includes all messages exchanged and is passed to the LLM in each request as part of the context. It resets when the session ends. Long-term memory persists across sessions and is typically implemented using vector databases (e.g., ChromaDB, Pinecone). When a new session starts, the agent retrieves relevant memories from the vector store using semantic search.

Q9: Why should you use temperature=0 when building agents that use tools?

ANSWER
Temperature controls the randomness of LLM outputs. At temperature=0, the model is deterministic — it always picks the most probable next token. For tool calling, you want consistent, predictable behavior. Higher temperatures introduce randomness that can cause the model to sometimes decide not to call a tool when it should, or to call the wrong tool. For creative tasks like writing, higher temperatures are appropriate; for agentic tasks, temperature=0 is the best practice.

10.3 Advanced Interview Questions

Q10: How would you design a multi-agent system for an e-commerce platform?

ANSWER
I would design specialized agents for different domains: (1) Catalog Agent — handles product search and recommendations using vector similarity. (2) Inventory Agent — checks stock levels by querying the database. (3) Order Agent — processes orders, returns, and refunds via the order management API. (4) Customer Agent — interfaces with the user and routes requests to specialized agents. (5) Analytics Agent — generates reports on sales and customer behavior. An Orchestrator coordinates all agents. Communication happens via message queues or a shared state object. Each agent has a specific set of tools and a domain-scoped prompt.

Q11: How do you handle agent hallucinations when using tool calling?

ANSWER
Tool calling significantly reduces hallucinations by grounding responses in real data. Additional strategies: (1) Instruct the LLM explicitly to only use information from tool results, not from its training knowledge. (2) Validate tool outputs before passing them to the LLM. (3) Use RAG (Retrieval-Augmented Generation) to ensure the agent has access to accurate, up-to-date documents. (4) Implement output validation — parse and verify the LLM's final response. (5) Log all tool calls and responses for human review. (6) Use guardrails frameworks like NVIDIA NeMo Guardrails for sensitive domains.

Q12: What is the challenge of long-running agents and how do you solve it?

ANSWER
Long-running agents face several challenges: (1) Context window limits — conversation history grows and eventually exceeds the LLM's context window. Solution: implement sliding window or summarization. (2) API costs — many tool calls and LLM invocations become expensive. Solution: caching, model selection (use cheaper models for simple steps). (3) Failures mid-task — a network error can halt the entire process. Solution: implement checkpointing to save intermediate state. (4) Infinite loops — agent keeps calling tools without converging. Solution: set max_iterations and implement loop detection.

10.4 HR & Conceptual Interview Questions

Q13: Why are you interested in AI Agents?

SUGGESTED ANSWER
AI Agents represent the next evolution in software engineering — moving from systems that respond to commands to systems that can autonomously accomplish goals. I am excited about the potential to build software that can handle complex, multi-step tasks that previously required human judgment. The combination of LLM reasoning with real-world tool access opens up possibilities that were not feasible just a few years ago. I see AI Agents becoming a core part of enterprise software within the next 2-3 years.

Q14: What are the ethical concerns with autonomous AI Agents?

ANSWER
Key ethical concerns include: (1) Unintended actions — agents with broad tool access could take harmful actions (deleting data, sending unauthorized emails). Mitigation: human-in-the-loop for critical actions, permission scoping. (2) Privacy — agents that read emails or access personal data raise privacy concerns.

Mitigation: data minimization, clear consent. (3) Accountability — when an agent causes harm, who is responsible? Mitigation: comprehensive audit logs. (4) Bias — LLMs can perpetuate biases in decision-making. Mitigation: diverse training data, bias audits. (5) Job displacement — automating tasks that humans currently perform. Mitigation: focus on augmentation rather than replacement.

11. Quick Revision & Cheat Sheet

11.1 Key Concepts at a Glance

Concept	One-Line Summary
AI Agent	Autonomous system that thinks, plans, and acts using LLM + Tools
LLM	The brain of the agent — understands language and makes decisions
Tool / Function	External capability the agent can invoke (API, database, calculator)
@tool decorator	Converts a Python function into a LangChain-compatible tool
bind_tools()	Registers tools with the LLM so it knows what's available
response.tool_calls	List of tool invocations the LLM wants to make
ReAct Pattern	Alternate Thought → Action → Observe loops until task is complete
Temperature=0	Use for deterministic, reliable tool calling behavior
load_dotenv()	Loads API keys from .env file into environment
try/except in tools	Always catch exceptions and return error strings, never raise

11.2 Complete Code Template — Copy and Start Building

```
# ——— AI AGENT TEMPLATE
# ——————————————————————————————————————————————————————————————

import requests
from langchain.tools import tool
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

# 1. Load environment variables
load_dotenv()

# 2. Define your tools
@tool
def your_tool_name(param: str) -> str:
    """Clear description of what this tool does and when to use it."""
    try:
        # Your tool logic here
        result = 'your result'
        return result
    except Exception as e:
        return f'Error: {e}'

# 3. Create LLM and bind tools
llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
llm_with_tools = llm.bind_tools([your_tool_name])

# 4. Run the agent
def run_agent(query: str):
```

```
    response = llm_with_tools.invoke(query)
    if response.tool_calls:
        tool_call = response.tool_calls[0]
        result = your_tool_name.invoke(tool_call['args'])
        print(f'Result: {result}')
    else:
        print(f'Response: {response.content}')

# 5. Test
run_agent('Your test query here')
# ──────────────────────────────────────────────────────────────────────────────
```

11.3 Important Commands Reference

Command / Syntax	Purpose
pip install requests langchain langchain-openai python-dotenv	Install all required packages
from dotenv import load_dotenv; load_dotenv()	Load API keys from .env file
@tool	Convert function to LangChain tool
llm.bind_tools([tool1, tool2])	Register tools with LLM
llm.invoke(query)	Send query to LLM
response.tool_calls	Check if LLM wants to call a tool
tool.invoke({'param': 'value'})	Execute a tool with arguments
response.content	Access LLM text response (when no tool call)
ChatOpenAI(model='gpt-4o-mini', temperature=0)	Create OpenAI LLM instance
requests.get(url, timeout=10).json()	Make API call and parse JSON

11.4 The 7 Golden Rules of AI Agent Development

13. Always write clear, specific docstrings for every tool — the LLM depends on them.
14. Always wrap tool code in try/except — never let tools crash the agent.
15. Always use temperature=0 for reliable tool calling.
16. Always store API keys in .env files — never hardcode credentials.
17. Always test tools independently before connecting them to the LLM.
18. Always set a max iteration limit to prevent infinite loops.
19. Always log tool calls and results for debugging and auditing.

11.5 Final One-Line Summary

AI Agents are intelligent systems that use LLMs as their brain and Tools as their hands — enabling them to automatically solve complex, real-world tasks.

— END OF STUDY MATERIAL —
Tech Craft | AI Agents | Day 6 | Comprehensive Training Notes